

be subjective, based first on the required features, then on optional features, and finally on general usability and look-and-feel. To ensure that I fairly evaluate your solution, you should include in your submission a list of any optional features you incorporated.

Three Months Ago Winner

After a Challenge absence of more than three years, **Jeff Mallett** (Boulder Creek, CA) returns to take first place in the June Dots Challenge. The object of this Challenge was to win a round-robin tournament of the game Dots (or Dots and Boxes). Dots is played on an NxN grid where players take turns connecting adjacent dots horizontally and vertically to enclose boxes. The player capturing the most boxes wins the game. The Challenge was actually scored based on minimizing the number of boxes captured by the opposing player, incorporating the usual efficiency requirement by adding a penalty of 1% for each millisecond of execution time. Solutions were also required to display the game state and the current score after each move.

Jeff maintains four doubly-linked lists of boxes in gList, one list for boxes with 0 edges (0-boxes), another for boxes with 1 edge (1-boxes), etc. He also maintains a data structure called a 2-path (TwoPathRecordType), which is a sequence of connected boxes each of which has two or three existing edges (2-boxes or 3-boxes, respectively).

The heavy lifting is done in the ComputerTurn routine, and I'll try to describe the logic. If there are no 3-boxes, the code looks to see if all unfilled boxes are 2-boxes. If so, any move is going to give a box to the opponent, so Jeff picks the move that gives the minimum away to the opponent. Otherwise, he selects a 0-box, or (as a second choice) a 1-box, provided it is "safe", where an "unsafe" box is one for which adding an edge creates a 3-box.

If there are 3-boxes, and there are safe places to move next, Jeff takes the 3-box and plays again. If the open edge of the 3-box is at the board boundary, he takes the box. Otherwise, he takes as many 3-boxes as he can while saving the moves that give the opponent a square ("handouts"). If a move that gives a handout captures half or more of the remaining boxes, Jeff makes that move. And finally, if forced, he makes a move that gives the opponent the smallest sequence of boxes.

As the comments in Jeff's code indicate, his solution is actually based on 15-year-old code, translated from Pascal into C/C++ for the Challenge. Jeff mentions that the time penalty in the problem caused him to significantly "dumb down" the program, removing enough of the look-ahead code to make it a fast, if mediocre, player.

The second-place entry, from Greg Sadetsky, is based (with permission) on a JavaScript program by UCLA Professor Thomas S. Ferguson. In addition to providing some very entertaining commentary that I wish we had the space to publish, Greg included the URL for the JavaScript code (<http://www.stat.ucla.edu/~tom/Games/dots&boxes.html>), as well as a page of links to other analyses of the game (http://dmoz.org/Games/Paper_and_Pencil/Dots_and_Boxes/).

The table below lists, for each of the solutions submitted, the number of cells captured by each solution in the tournament, the number of cells captured by the opposing player, the execution time in milliseconds, and the score earned by each solution (with lower scores being better). The table also includes the code and data size for each solution, and the programming language used. As usual, the number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges prior to this one.

Name	Player Opponent		Time
	Cells	Cells	(msec)
Jeff Mallett (94)	3218	1362	833.7
Greg Sadetsky (14)	3128	1452	1566.6
Ernst Munter (751)	2529	2010	721.4
Tom Saxton (185)	1914	2666	3262.5
Randy Boring (142)	668	3422	53118.1
T. R.	1752	2297	367.1
Name	Score	Code	Data Lang
Jeff Mallett	2061.5	11572	908 C++
Greg Sadetsky	2753.8	17192	1936 C
Ernst Munter	3009.9	10684	586 C++
Tom Saxton	8777.5	7220	581 C++
Randy Boring	145423.8	17676	498 C
T. R.	2962.6	5188	307 C++

Top Contestants ...

Listed here are the Top Contestants for the Programmer's Challenge, including everyone who has accumulated 20 or more points during the past two years. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank	Name	Points (24 mo)	Wins (24 mo)	Total Points
1.	Munter, Ernst	271	10	758
2.	Rieken, Willeke	73	3	134
3.	Saxton, Tom	71	2	189
4.	Taylor, Jonathan	56	2	56
5.	Wihlborg, Claes	49	2	49
6.	Shearer, Rob	48	1	62
7.	Maurer, Sebastian	38	1	108

10.	Mallett, Jeff	20	1	114
11.	Truskier, Peter	20	1	20

... and the Top Contestants Looking for a Recent Win

In order to give some recognition to other participants in the Challenge, we also list the high scores for contestants who have accumulated points without taking first place in a Challenge during the past two years. Listed here are all of those contestants who have accumulated 6 or more points during the past two years.

Rank	Name	Points (24 mo)	Total Points
8.	Boring, Randy	32	144
9.	Sadetsky, Gregory	22	24
12.	Schotsman, Jan	14	14
13.	Nepsund, Ronald	10	57
14.	Day, Mark	10	30
15.	Jones, Dennis	10	22
16.	Downs, Andrew	10	12
17.	Desch, Noah	10	10
18.	Duga, Brady	10	10
19.	Fazekas, Miklos	10	10
20.	Flowers, Sue	10	10
21.	Strout, Joe	10	10
22.	Nicolle, Ludovic	7	55
23.	Hala, Ladislav	7	7
24.	Leshner, Will	7	7
25.	Miller, Mike	7	7
26.	Widyatama, Yudhi	7	7
27.	Heithcock, JG	6	43

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points
2nd place	10 points
3rd place	7 points
4th place	4 points
5th place	2 points
finding bug	2 points
suggesting Challenge	2 points

Here is Jeff's winning Boxes solution:

Boxes.cpp

Copyright © 1986-2001

Jeff Mallett

```
// Dots & Boxes
//
// Copyright 1986-2001 Jeff Mallett. All rights reserved.
// For licensing contact jeffm@zillions-of-games.com
//
// Written in Lightspeed Pascal for the Mac
// Last revision in Pascal: Oct 14, 1986
// Ported from Pascal to C/C++ and adapted for contest: May-June 2001
// (It uses inline/new/delete/template/etc., but it's really more C
// than C++ since there are no classes. If I had more time I would
// have converted it to be object-oriented as well.)
//
// Programmer's Challenge Entry
// This version does not play Dots & Boxes completely naively - e.g.
// it understands the sacrificing boxes to keep the initiative - but
// neither does it play very skillfully. This is because the
// challenge awards points based on boxes and time taken, not
// on who actually wins the games. As penalties are awarded
// for every millisecond of thinking, a quick mediocre player is
// likely to gain more points than a slow master. Therefore I
// dumbed down the program considerably by ripping out code that
// determined how to correctly play networks of 1-boxes
// connected with 2-paths of length 1 or 2. Now the program
// doesn't have a clue what to do with unsafe 1-boxes and never
```

```

// sacrifices boxes early in the game, but it also saves a
// lot of processing time.
//

#define DRAWING
#define EDGECOUNT_FIELD
// #define COORDINATE_FIELDS

#define TICKSEED LMGetTicks()
#define ASSERT assert

#include
#include
#include

#include "Dots.h"

#include

ENUMS
enum direction {left=0, up, right, down};
enum who {firstplayer=0, secondplayer};
enum pathdirection {behind=0, ahead};
// BORDER_VALUE: box is off the board
// MARK_VALUE: when calculating a path, a box with this value is already on it
enum boxflag {BORDER_VALUE=-2, MARK_VALUE=-1, NO_VALUE=0};
// NOT_LOOP: 2-path has two ends
// ALMOST_LOOP: 2-path's ends terminate at the same 0-box or 1-box
// LOOP: 2-path has no ends
enum loopvalue {NOT_LOOP=0, ALMOST_LOOP, LOOP};

TYPEDEFS
typedef struct BoxRecordType* PtrToBoxRecord;
typedef struct TwoPathRecordType* PtrToTwoPathRecord;

struct TwoPathRecordType {
    int pathsize;
    loopvalue loop;
    PtrToBoxRecord path;
    PtrToTwoPathRecord previous, next;
};

struct BoxRecordType {
#ifdef COORDINATE_FIELDS
    short xcoord, ycoord;           // Location of box
#endif
#ifdef EDGECOUNT_FIELD
    short edgecount;               // Number of edges
#endif
    Boolean edges[4];               // Is there an edge in this direction?
    PtrToBoxRecord previous, next; // previous & next elements in gList[]

    boxflag flag;                  // Used for marking borders and searched boxes
// 2-path info
    PtrToTwoPathRecord twopath;    // the 2-path this is in
    PtrToBoxRecord pnext[2];       // the two connecting boxes in the 2-path
};

struct MoveRecordType {
    PtrToBoxRecord box;
    direction dir;
};

GLOBALS
PtrToBoxRecord gBoxes;           // Array holding all the boxes
int gArraySizeY;                 // Array width of gBoxes
int gBoardSizeX, gBoardSizeY;    // Size of the board in boxes across and down
PtrToBoxRecord gList[4];         // Pointers to doubly-linked lists of n-edge boxes
PtrToBoxRecord gListBookmark[2]; // Next box on gList[n] to look at for a safe move
int gOffset[4];                  // Offset in gBoxes for going this direction

PtrToTwoPathRecord gTwoPaths[3]; // Pointers to a doubly-linked lists of 2-paths
// [0]:length > 2 [1]:length 1 [2]:length 2
Boolean gPathsComputed;          // Have the 2-paths ever been recorded in gTwoPaths?

Boolean gSafe[2];
// Is there a move on an n-box that doesn't give the opponent a 3-box?
Boolean gSafetyCheckNeeded;
// Need to check whether position has become dangerous?
PtrToBoxRecord gLastSafeBox[2]; // The most recent box found safe on list n

int gScore[2];                   // gScore[w] is the # of boxes player w has completed
int gTotalScore;                 // The sum of the two player's score
int gMaxScore;                   // The maximum possible value of gTotalScore
who gCurrentPlayer;              // The current player

MACROS

```

```

#define EVEN(x)                (((x) & 1) == 0)
#define ODD(x)                 (((x) & 1) != 0)

#define EDGE(x,y,d)            gBoxes[x*gArraySizeY + y].edges[d]
#define BOX(x,y)               (&gBoxes[x*gArraySizeY + y])

#define FOR_LIST(pList, whichList)  for (pList = gList[whichList]; pList; pList = pList->next)

#define FOR_LIST2(pList, whichList, start) \
    if (gList[whichList]) { \
        if (!start || NumberOfEdges(start) != whichList) \
            start = gList[whichList]; \
        pList = start; do {

#define END_FOR_LIST2(pList, whichList, start) \
        pList = pList->next; \
        if (!pList) pList = gList[whichList]; \
    } while (pList != start); \
    }

#define FOR_EACH_OPEN_DIRECTION(box) \
{ direction f_dir = left; const Boolean *pEdges = (box)->edges; \
    do { \
        if ( !(pEdges + (f_dir)) ) { \
            PtrToBoxRecord f_box = Go(box, f_dir);

#define END_FOR_EACH_OPEN_DIRECTION \
    } \
    } while ((f_dir = (direction)(f_dir+1)) <= down); \
}

#define FOR_PATHDIRECTION(pdir)    pdir = behind; do
#define END_FOR_PATHDIRECTION(pdir) while ((pdir = (pathdirection)(pdir+1)) <= ahead);

PROTOTYPES
#ifdef DRAWING
extern void DrawInitial(int x, int y, who person);
extern void DrawScore(who w);
extern void DrawMove(int x, int y, enum direction d);
extern void InitialDrawing();
#endif

void Initialize();

void CheckSafety(int listindex);
void Clear(PtrToBoxRecord box);
void ComputeAPath(PtrToBoxRecord posinedgelist, PtrToTwoPathRecord existingtwopath);
void ComputePaths();
void UpdatePathFrom(PtrToBoxRecord box);
void AddEdge(PtrToBoxRecord box, direction d, who whoseturn);
int ExpectedScore(PtrToTwoPathRecord ignoreThis);
int CountTinyTwoPaths( );
int CountSurroundingPathSizes(PtrToBoxRecord box);
PtrToTwoPathRecord MinimumTwoPath();
void HandOut(int hotype, MoveRecordType &move);
direction HardHeartedHandout(PtrToBoxRecord box);
Boolean FindSafeMoveOnList(int listindex, MoveRecordType &move);
void ComputerTurn(MoveRecordType &move);
Boolean MakeRealMove(MoveRecordType move, who whoseturn);
void ConvertFromDotLine(MoveRecordType &move, const DotLine &dotline);
void ConvertToDotLine(MoveRecordType move, DotLine &dotline);

INLINES
// Returns X coordinate value for the box
inline int GetX (PtrToBoxRecord box) {
#ifdef COORDINATE_FIELDS
    return box->xcoord;
#else
    return (box - gBoxes) / gArraySizeY;
#endif
}

// Returns Y coordinate value for the box
inline int GetY (PtrToBoxRecord box) {
#ifdef COORDINATE_FIELDS
    return box->ycoord;
#else
    int dindex = box - gBoxes;
    int xcoord = dindex / gArraySizeY;
    return dindex - xcoord * gArraySizeY;
#endif
}

// Returns the box in direction dir
inline PtrToBoxRecord Go (PtrToBoxRecord box, direction dir) {
    return box + gOffset[dir];
}

```

```

// Returns the opposite path direction to the given path direction
inline pathdirection OppositePathDirection (pathdirection pathdir) {
    return (pathdirection) (1 - pathdir);
}

// Returns the direction opposite to the given direction
inline direction OppositeDirection (direction dir) {
    return (direction) ((dir+2)%4);
}

// Returns the next direction in a clockwise fashion
inline direction NextDirection (direction dir) {
    return (dir == down) ? left : (direction) (dir + 1);
}

// Goes to the next player
inline void NextPlayer (who &whoseturn) {
    whoseturn = (who) (1 - whoseturn);
}

// Produces a random number between 0 and n-1 inclusive
inline int rnd (int n) {
    return ((long)rand() * n) / ((long)RAND_MAX + 1);
}

// Returns a direction chosen at random (or at least different)
inline direction RandomDirection () {
    return (direction) (rand() & 3);
}

// Given a 3-box on a 2-path, returns the path direction out of the 3-box
inline pathdirection OutPathDirection (PtrToBoxRecord box) {
    return box->pnext[ahead] ? ahead : behind;
}

// Returns true iff the box is within the boundaries.
inline Boolean IsInsideBoard (PtrToBoxRecord box) {
    return box->flag != BORDER_VALUE;
}

// Returns a direction randomly such that !box->edges[d]
inline direction GetOpenDirection (PtrToBoxRecord box) {
    // cycle through directions starting at random direction dir
    direction d = RandomDirection();
    const Boolean *p = box->edges;
    while (*(p+d))
        d = NextDirection(d);
    return d;
}

// Sets up a move from box in a random direction with no edge
inline void PlayAny (PtrToBoxRecord box, MoveRecordType &move) {
    move.box = box;
    move.dir = GetOpenDirection(box);
}

// Plays a move on the path p
inline void PlayMoveOnPath (PtrToTwoPathRecord p, MoveRecordType &move) {
    move.box = p->path;
    move.dir = (p->pathsize == 2) ?
        HardHeartedHandout(move.box) : GetOpenDirection(move.box);
}

// Returns whether or not boxes a and b are connected
inline Boolean AreConnected (PtrToBoxRecord a, PtrToBoxRecord b) {
    FOR_EACH_OPEN_DIRECTION(a)
    {
        if (f_box == b)
            return true;
    } END_FOR_EACH_OPEN_DIRECTION;
    return false;
}

// Returns the number of edges of the box
inline int NumberOfEdges (PtrToBoxRecord box) {
#ifdef EDGECOUNT_FIELD
    return box->edgecount;
#else
    int count = 0;
    Boolean *p = box->edges;
    if (*(p++)) ++count;
    if (*(p++)) ++count;
    if (*(p++)) ++count;
    if (*(p++)) ++count;
    if (*p) ++count;
    return count;
#endif
}

```

LIST TEMPLATES

```
// Adds the element to the beginning of the given list
template<class T> void AddToList(T element, T &list) {
    element->previous = NULL;
    element->next = list;
    list = element;
    if (element->next)
        element->next->previous = element;
}

// Removes the element from the given list
template<class T> void RemoveFromList(T element, T &list) {
    if (element->previous)
        element->previous->next = element->next;
    else
        list = element->next;
    if (element->next)
        element->next->previous = element->previous;
}

// Inserts toinsert into the list after the onlist box
// Note: toinsert must not be on a list currently
template<class T> void InsertAfter(T toinsert, T onlist) {
    if (onlist->next)
        onlist->next->previous = toinsert;
    toinsert->next = onlist->next;
    onlist->next = toinsert;
    toinsert->previous = onlist;
}

// Adds path to the front of the appropriate gTwoPaths list
inline void AddToPathsList(PtrToTwoPathRecord path) {
    AddToList(path, gTwoPaths[path->pathsize <= 2 ? path->pathsize : 0]);
}

// Removes path from the appropriate gTwoPaths list
inline void RemoveFromPathsList(PtrToTwoPathRecord path) {
    RemoveFromList(path, gTwoPaths[path->pathsize <= 2 ? path->pathsize : 0]);
}

Initialize
// Sets up variables for a game.
void Initialize()
{
    int i, j;
    int arraySizeX, arrayElements;

    srand(TICKSEED);

    arraySizeX = gBoardSizeX + 2; // borders on each side
    gArraySizeY = gBoardSizeY + 2;
    arrayElements = arraySizeX * gArraySizeY;
    gBoxes = new BoxRecordType [arrayElements];
    ASSERT(gBoxes);

    gOffset[left] = -gArraySizeY;
    gOffset[up] = -1;
    gOffset[right] = gArraySizeY;
    gOffset[down] = 1;

    PtrToBoxRecord p = gBoxes;
    for (i = 0; i < arraySizeX; i++)
        for (j = 0; j < gArraySizeY; j++)
        {
            p->edges[left] = p->edges[up] = p->edges[right] =
                p->edges[down] = false;
            p->flag = BORDER_VALUE;
            p->pnext[ahed] = p->pnext[behind] = NULL;
            p->twopath = NULL;
#ifdef COORDINATE_FIELDS
            p->xcoord = i; p->ycoord = j;
#endif
#ifdef EDGECOUNT_FIELD
            p->edgecount = 0;
#endif
            ++p;
        }

    gList[0] = BOX(1,1);
    gList[0]->previous = NULL;

    for (i = 1; i <= 3; i++)
        gList[i] = NULL;

    PtrToBoxRecord last = NULL;
    for (i = 1; i <= gBoardSizeX; i++)
    {
        p = BOX(i, 1);
```

```

    for(j = 1; j <= gBoardSizeY; j++)
    {
        p->flag = NO_VALUE;
        if (last)
        {
            last->next = p;
            p->previous = last;
        }
        last = p;
        ++p;
    }
}
last->next = NULL;

// Mix up list elements randomly
for (i = gBoardSizeX; i >= 1; i --)
    for (j = 1; j <= gBoardSizeY; j ++)
    {
        int i2 = rnd(gBoardSizeX)+1;
        int j2 = rnd(gBoardSizeY)+1;
        if (i!=i2 || j!=j2)
        {
            RemoveFromList(BOX(i, j), gList[0]);
            InsertAfter(BOX(i, j), BOX(i2, j2));
        }
    }
}
ASSERT(!gList[0]->previous);

gSafe[0] = gSafe[1] = true;
gPathsComputed = gSafetyCheckNeeded = false;
gMaxScore = gBoardSizeX * gBoardSizeY;
gTotalScore = gScore[firstplayer] = gScore[secondplayer] = 0;
gListBookmark[0] = gListBookmark[1] =
    gLastSafeBox[0] = gLastSafeBox[1] = NULL;
} // Initialize

```

ENGINE

CheckSafety

```

// Re-evaluates gSafe
// If we can't add an edge to a 0-box or 1-box without changing
// a 2-box into a 3-box set gSafe[listindex]=false
void CheckSafety(int listindex) {

```

```

    // Optimization: Check the box that was found safe last time
    if (gLastSafeBox[listindex])
    {
        if (NumberOfEdges(gLastSafeBox[listindex]) <= 1)
        {
            FOR_EACH_OPEN_DIRECTION(gLastSafeBox[listindex])
            {
                if (NumberOfEdges(f_box) != 2)
                    return; // safe
            } END_FOR_EACH_OPEN_DIRECTION;
        }
        gLastSafeBox[listindex] = NULL;
    }
}

```

```

    // Check 0 and 1 lists
    PtrToBoxRecord box;
    FOR_LIST(box, listindex)
    {
        // Check a box on list
        FOR_EACH_OPEN_DIRECTION(box)
        {
            if (NumberOfEdges(f_box) != 2)
            {
                gLastSafeBox[listindex] = box;
                return;
            }
        } END_FOR_EACH_OPEN_DIRECTION;
    }
}

```

```

    gSafe[listindex] = false; // unsafe
} // CheckSafety

```

Clear

```

// Initializes/Reinitializes the path data for a single record
void Clear (PtrToBoxRecord box) {

```

```

    PtrToTwoPathRecord p = box->twopath;
    if (p)
    {
        RemoveFromPathsList(p);
        if (!p->pathsize)
        { // Box is only thing on twopath
            delete p;
        } else
        {

```

```

    AddToPathsList(p);
    if (p->path == box)
    {
        ASSERT(box->pnext[ahead]);
        p->path = box->pnext[ahead];
    }
    pathdirection pdir = OutPathDirection(box);
    box->pnext[pdir]->pnext[OppositePathDirection(pdir)] = NULL;
}
box->twopath = NULL;
box->pnext[ahead] = box->pnext[behind] = NULL;
}
} // Clear

ComputeAPath
// Given a 2-box or a 3-box, computes the path from that box
void ComputeAPath (PtrToBoxRecord posinedgelist, PtrToTwoPathRecord existingtwopath) {

    Boolean pathend;
    PtrToBoxRecord temp;
    pathdirection pdir;
    int pathlength = -1; // set to -1 because posinedge is counted twice
    PtrToBoxRecord end[2], beyondend[2];
    beyondend[ahead] = beyondend[behind] = NULL;
    loopvalue loop = NOT_LOOP;
    PtrToTwoPathRecord twopath;

    if (existingtwopath)
    {
        twopath = existingtwopath;
        RemoveFromPathsList(twopath);
    } else
    {
        twopath = new TwoPathRecordType;
        ASSERT(twopath);
    }

    FOR_PATHDIRECTION(pdir)
    {
        // each iteration moves to the end of the path in the behind/ahead direction
        temp = posinedgelist;
        do { // each iteration marks one more square in the path
            pathlength++;
            temp->flag = MARK_VALUE;
            temp->twopath = twopath;
            pathend = true;
            // check for a continuation of the path in the pdir direction
            FOR_EACH_OPEN_DIRECTION(temp)
            {
                // each iteration checks one direction for an adjoining 2/3-square
                // don't need to check if f_box inside board because borders have
                // BORDER_VALUE
                if (!f_box->flag)
                {
                    // found square next to current square that hasn't been marked
                    int n = NumberOfEdges(f_box);
                    if (n >= 2)
                    {
                        // include this square in path
                        temp->pnext[pdir] = f_box;
                        f_box->pnext[OppositePathDirection(pdir)] = temp;
                        temp = f_box;
                        if (n == 2)
                        {
                            pathend = false;
                        }
                        else // n == 3
                        {
                            // f_box is on path so add it, but don't look further
                            pathlength++;
                            temp->flag = MARK_VALUE;
                            temp->twopath = twopath;
                        }
                    }
                    break;
                }
                // n < 2
                if ( pdir == ahead || temp != posinedgelist )
                {
                    beyondend[pdir] = f_box;
                    break;
                }
            }
        }
        // END_FOR_EACH_OPEN_DIRECTION;
        // Either all directions have been looked at or a path continuation has
        // been found
    } while (!pathend);
    temp->pnext[pdir] = NULL;
    end[pdir] = temp;
} END_FOR_PATHDIRECTION(pdir);

// Clear flags
for (temp = end[ahead]; temp; temp = temp->pnext[behind])
    temp->flag = NO_VALUE;

```

```

// Is a loop or near loop?
if (pathlength > 2)
{
    if (beyondend[ahead] && beyondend[ahead] == beyondend[behind])
        loop = ALMOST_LOOP;
    else if (AreConnected(end[ahead], end[behind]))
        loop = LOOP;
}

twopath->pathsize = pathlength;
twopath->loop = loop;
twopath->path = end[behind];
AddToPathsList(twopath);
} // ComputeAPath

ComputePaths
// Finds all 2-paths and stores them by forming a doubly-linked list for each path
void ComputePaths() {

    PtrToBoxRecord posinedgelist;

    FOR_LIST(posinedgelist, 2)
    {
        if (!posinedgelist->twopath)
            ComputeAPath(posinedgelist, NULL);
        // Compute path containing this element
    }
    gPathsComputed = true;
} // ComputePaths

UpdatePathFrom
// Updates the paths that the newly changed box was in if needed
void UpdatePathFrom (PtrToBoxRecord box) {

    int count;

    // Update path lists
    count = NumberOfEdges(box);
    if (count == 4)
        Clear(box);
    else if (count == 2)
    {
        // Could be added to a 2-path
        PtrToTwoPathRecord twopath = NULL;
        FOR_EACH_OPEN_DIRECTION(box)
        {
            if (f_box->twopath)
            {
                if (!twopath)
                    twopath = f_box->twopath; // Recalculate this 2-path
                else if (f_box->twopath != twopath)
                { // Combining two 2-paths
                    RemoveFromPathsList(f_box->twopath);
                    break;
                }
            }
        }
        END_FOR_EACH_OPEN_DIRECTION;
        ComputeAPath(box, twopath);
    }
    else if (count == 3 &&
        !(box->twopath && box->twopath->pathsize == 1))
        // don't need to update a length 1 path
        ComputeAPath(box, box->twopath);
} // UpdatePathFrom

AddEdge
// Adds an edge to box at x,y and direction d. If the edge completes a square
// update score, etc.
// If safe might need to be changed then checkneeded will be set to true.
void AddEdge (PtrToBoxRecord box, direction d, who whoseturn) {

    box->edges[d] = true;
#ifdef EDGE_COUNT_FIELD
    ++box->edgecount;
#endif

    if (!IsInsideBoard(box))
        return;

    int count = NumberOfEdges(box);
    if (count == 4)
    {
        gScore[whoseturn]++;
#ifdef DRAWING
        DrawScore(whoseturn);
        DrawInitial(GetX(box), GetY(box), whoseturn);
#endif
    }
    ++gTotalScore;
    RemoveFromList(box, gList[3]); // Take box off of old edge list

```

```

    // Don't bother putting it on gList[4]
}
else
{
    RemoveFromList(box, gList[count-1]); // Take box off of old edge list
    AddToList(box, gList[count]); // Add box to the beginning of a new list

    if (count == 2)
        gSafetyCheckNeeded = true;
}
} // AddEdge

```

ExpectedScore

// Return estimate of the number of boxes we'll capture

int ExpectedScore(PtrToTwoPathRecord ignoreThis) {

ASSERT(gPathsComputed);

int expected, tinycount1, tinycount2, loopcount[3];

loopcount[NOT_LOOP] = loopcount[ALMOST_LOOP] =
 loopcount[LOOP] = tinycount1 = tinycount2 = expected = 0;

// Count

PtrToTwoPathRecord p;

for (p = gTwoPaths[1]; p; p = p->next)

++tinycount1;

for (p = gTwoPaths[2]; p; p = p->next)

++tinycount2;

for (p = gTwoPaths[0]; p; p = p->next)

```

{
    expected += p->pathsize; // Add boxes for 2-paths
    ++loopcount[p->loop];
}

```

// Subtract out ignoreThis

if (ignoreThis->pathsize == 1)

-tinycount1;

else if (ignoreThis->pathsize == 2)

-tinycount2;

else {

expected -= ignoreThis->pathsize;

-loopcount[ignoreThis->loop];

}

if (loopcount[NOT_LOOP] || loopcount[ALMOST_LOOP] ||
 loopcount[LOOP])

```

{
    // Subtract handouts
    expected -= loopcount[NOT_LOOP] * 2;
    expected -= loopcount[ALMOST_LOOP] * 3;
    expected -= loopcount[LOOP] * 4;

```

// but disregard final handout

expected += loopcount[NOT_LOOP] ? 2 : 4;

}

// Add boxes for tiny 2-paths

expected += tinycount1/2 + 2 * (tinycount2/2);

// Get half of each rounded down

if (ODD(tinycount1) && ODD(tinycount2))

expected += 2; // Get last [2], e.g. 1 [1] and 1 [2]

return expected;

} // ExpectedScore

CountTinyTwoPaths

// Counts length=1 2-paths and length=2 2-paths (no 3's).

int CountTinyTwoPaths () {

int count = 0;

PtrToTwoPathRecord p;

for (p = gTwoPaths[1]; p; p = p->next)

if (NumberOfEdges(p->path) == 2)

++count;

for (p = gTwoPaths[2]; p; p = p->next)

if (NumberOfEdges(p->path) == 2 &&
 NumberOfEdges(p->path->pnext[ahead]) == 2)

++count;

return count;

} // CountTinyTwoPaths

CountSurroundingPathSizes

// Returns the number of boxes on all the surrounding

// 2-paths. (Some may be counted more than once.)

int CountSurroundingPathSizes(PtrToBoxRecord box) {

```

int count = 0;
FOR_EACH_OPEN_DIRECTION(box)
{
    if (f_box->twopath)
        count += f_box->twopath->pathsize;
} END_FOR_EACH_OPEN_DIRECTION;
return count;
} // CountSurroundingPathSizes

MinimumTwoPath
// Returns the smallest 2-path.
PtrToTwoPathRecord MinimumTwoPath () {

    ASSERT(!gList[3]);

    if (! gPathsComputed)
        ComputePaths();

    int minvalue = INT_MAX;
    PtrToTwoPathRecord min,    p;

    if (gTwoPaths[1])
        return gTwoPaths[1];

    if (gTwoPaths[2])
        return gTwoPaths[2];

    for (p = gTwoPaths[0]; p; p = p->next)
    {
        int size = p->pathsize;
        if (p->loop != LOOP)
        {
            size += 3; // Penalize 2 since only get a 2-handout instead of a 4-handout
            // Also give 1 penalty since we'd much rather be left with non-loop
            // than a loop at the very end, because we won't get the last handout.

            if (p->loop == ALMOST_LOOP && size < minvalue)
            {
                PtrToBoxRecord box = p->path;
                FOR_EACH_OPEN_DIRECTION(box)
                {
                    if (NumberOfEdges(f_box) == 1)
                    {
                        // f_box is connected to both edges of this 2-path
                        // plus one other 2-path. Filling in this 2-path
                        // will cause the two 2-paths to be connected, so
                        // the size should be calculated as the total of
                        // both paths + 1 for the 1-box + 3 for the penalty
                        // (size is subtracted because the almost loop is
                        // counted twice)
                        size = 4 + CountSurroundingPathSizes(f_box) - size;
                        break;
                    }
                }
            } END_FOR_EACH_OPEN_DIRECTION;
        }

        if (size < minvalue)
        {
            minvalue = size;
            min = p;
            if (size == 3)
                break;
        }
    }

    return min;
} // MinimumTwoPath

```

```

HandOut
// Given a HandOut type (either 2 or 4) and that every box on the 3's list is
// either a 2 or hand out, HandOut finds a hand out of the indicated type in
// the 3's list and sets move to do this hand out.
void HandOut (int ho_type, MoveRecordType &move) {

```

```

    PtrToBoxRecord temp;
    pathdirection pdir;

    FOR_LIST(temp, 3)
    {
        if (temp->twopath->pathsize == ho_type)
        {
            // HandOut
            pdir = OutPathDirection(temp);
            move.box = temp->pnext[pdir];
            FOR_EACH_OPEN_DIRECTION(move.box)
            {
                if (!IsInsideBoard(f_box) || temp != f_box)

```

```

        {
            move.dir = f_dir;
            return;
        }
    } END_FOR_EACH_OPEN_DIRECTION;
    break;
} // if
}
} // HandOut

HardHeartedHandout
// Given a 2-box on a 2-path of length two, returns the
// correct direction that a new edge should be placed.
direction HardHeartedHandout (PtrToBoxRecord box) {

    FOR_EACH_OPEN_DIRECTION(box)
    {
        if (NumberOfEdges(f_box) == 2)
            return f_dir;
    } END_FOR_EACH_OPEN_DIRECTION;
    ASSERT(0); // should never get here
    return left;
} // HardHeartedHandout

FindSafeMoveOnList
// Tries to find a safe move on gList[listindex]
// If so, set move and return true
// If not, these boxes are now unsafe. Return false
Boolean FindSafeMoveOnList(int listindex, MoveRecordType &move) {

    PtrToBoxRecord box;

    // If any legal adjacent element is !=2 or outside board, take it
    FOR_LIST2(box, listindex, gListBookmark[listindex])
    {
        direction k, d;
        d = k = RandomDirection();
        do {
            if (!box->edges[d])
            {
                PtrToBoxRecord box2 = Go(box, d);
                if (!IsInsideBoard(box2) || NumberOfEdges(box2) != 2)
                {
                    gListBookmark[listindex] = box->next;
                    move.box = box;
                    move.dir = d;
                    return true;
                }
            }
        } while (d != k);
    } END_FOR_LIST2(box, listindex, gListBookmark[listindex])

    gSafe[listindex] = false;
    return false;
} // FindSafeMoveOnList

ComputerTurn
// Come up with a move for the computer
void ComputerTurn(MoveRecordType &move) {

    PtrToBoxRecord temp;

    // generate computer's move
    if (!gList[3])
    {
        if (!gList[0] && !gList[1])
        {
            // must pick a 2-box from a 2-path
            PlayMoveOnPath(MinimumTwoPath(), move);
            return;
        }

        // (gList[0]!=NULL or gList[1]!=NULL) and !gList[3]

        // Try safe boxes on lists
        if (gSafe[0] && FindSafeMoveOnList(0, move))
        {
            return;
        }
        if (gSafe[1] && FindSafeMoveOnList(1, move))
        {
            return;
        }

        // All bad choices: all 0's and 1's are adjacent to 2's
        if (!gPathsComputed)
            ComputePaths();
    }
}

```

```

    // now count how many 2's in a row- if one, take it
    // - if two, take it (put | between 2's)
    PlayMoveOnPath(MinimumTwoPath(), move);
    return;
}

// gList[3]!=NULL
if (gSafetyCheckNeeded)
{
    if (gSafe[0])
        CheckSafety(0);
    if (gSafe[1])
        CheckSafety(1);
    gSafetyCheckNeeded = false;
}

if (gSafe[0] || gSafe[1] || !gList[2] ||
    gMaxScore - gTotalScore <= 4)
{
    // Note: If the number of untaken boxes is <= 4, then always
    // take a box, since hand-outs can't be better
    PlayAny(gList[3], move);
    return;
}

if (!gPathsComputed)
    ComputePaths();

// unsafe && gList[2]!=NULL && gList[3]!=NULL
// while not at end of 3's list do
//   if the box connected to the 3-box is not a 2-box (or is outside the board)
//   then take it
FOR_LIST(temp, 3)
{
    if (!temp->twopath)
    {
        PlayAny(temp, move);
        return;
    }
}

// unsafe & gList[3]!=NULL & all 3-boxes are part of 2-lists
// Take all 3's that will not ruin HandOut possibilities
FOR_LIST(temp, 3)
{
    // if connecting 2-path is not of length 2 or 4 then take it
    ASSERT(temp->twopath);
    int size = temp->twopath->pathsize;
    if (size != 2 && size != 4)
    {
        PlayAny(temp, move);
        return;
    }

    // if 2-path looks like 3-2-2 then take it
    if (size == 4)
    {
        pathdirection pdir = OutPathDirection(temp);
        PtrToBoxRecord box2 =
            temp->pnext[pdir]->pnext[pdir]->pnext[pdir];
        if (NumberOfEdges(box2) == 2)
        {
            PlayAny(temp, move);
            return;
        }
    }
}

// Now all 3-paths look like 3-2 or 3-2-2-3. These both are
// are hand out possibilities...
//
//      2-path          |__| -> |__|
//
//      4-path          |__|__| -> |__|__|
//
// Before we hand out though, let's see if there are any
// tiny 2-paths where 2 or 4 handouts don't exist.
// These change the initiative.
if (!gList[1])
{
    // special unsafe 1-box configurations are not counted
    int tiny2paths = CountTinyTwoPaths ();
    if (ODD(tiny2paths))
    {
        // We do not need to retain the initiative because it will
        // change in our favor anyway
        PlayAny(gList[3], move);
        return;
    }
}

```

```

}

// It is possible to have more than one HandOut available so
// count the number of each kind of HandOut possibility.
int num2ho, // the number of length 2 hand out possibilities
    num4ho; // the number of length 4 hand out possibilities
num2ho = num4ho = 0;
FOR_LIST(temp, 3)
    if (temp->twopath->pathsize == 2)
        num2ho++;
    else
        num4ho++;
num4ho /= 2; // were counted twice
if (num2ho + num4ho > 1)
{
    // More than one hand out available so grab another square first
    if (num4ho)
    {
        // take the box from a length 4 2-path
        temp = gList[3];
        assert(temp->twopath);
        while (temp->twopath->pathsize != 4)
            temp = temp->next;
        PlayAny(temp, move);
        return;
    }

    // more than one length=2 hand outs available so play one
    PlayAny(gList[3], move);
    return;
}

// only one handout possibility

// Now see if it's worth it, or we should just be greedy.
if (ExpectedScore(gList[3]->twopath) * 2 <
    gMaxScore - gTotalScore)
{
    // It's not worth it: just be greedy and don't give handout
    PlayAny(gList[3], move);
    return;
}

if (num2ho)
{
    // do a TPH
    HandOut(2, move);
    return;
}

ASSERT(num4ho == 1);
// do a FPH (ickk)
HandOut(4, move);
} // ComputerTurn

MakeRealMove
// Makes a move on the board. Returns true if a new block was formed.
Boolean MakeRealMove(MoveRecordType move, who whoseturn) {

    // Take care of the player's move
#ifdef DRAWING
    DrawMove(GetX(move.box), GetY(move.box), move.dir);
#endif
    int oldscore = gScore[whoseturn];
    // score of the current player before his move
    Boolean checkneeded = false;

    AddEdge(move.box, move.dir, whoseturn);

    PtrToBoxRecord box2 = Go(move.box, move.dir);
    AddEdge(box2, OppositeDirection(move.dir), whoseturn);

    if (gPathsComputed)
    {
        Boolean same2path = box2->twopath &&
            (move.box->twopath == box2->twopath) &&
            NumberOfEdges(move.box) == 3;
        if (same2path)
            box2->twopath = NULL;
        // See if updating move.box's 2-path updates this too
        UpdatePathFrom(move.box);

        // No need to update the path again if the boxes are still on the same 2-path
        if (!same2path || !box2->twopath)
            // Note that we want box2's 2-path to remain NULL if set above,
            // since that will cause a new 2-path to be created
            UpdatePathFrom(box2);
    }

    return oldscore != gScore[whoseturn]; // did score change?
} // MakeRealMove

```

CHALLENGE INTERFACE

// Converts from input data types to engine data types. Modifies box,d
void ConvertFromDotLine(MoveRecordType &move, const DotLine &dotline) {

```
    int x = dotline.dot1.col + 1;
    int y = dotline.dot1.row + 1;

    if (x > gBoardSizeX)
    {
        -x;
        move.dir = right;
    }
    else if (y > gBoardSizeX)
    {
        -y;
        move.dir = down;
    }
    else if (dotline.dot2.col > dotline.dot1.col)
        move.dir = up;
    else // (dotline.dot2.row > dotline.dot1.row)
        move.dir = left;

    move.box = BOX(x, y);
}
```

// Converts from engine data types to output data types, Modifies dotline
void ConvertToDotLine(MoveRecordType move, DotLine &dotline) {

```
    int x = GetX(move.box);
    int y = GetY(move.box);
    direction d = move.dir;

    if (d == right)
    {
        d = left;
        ++x;
    }
    else if (d == down)
    {
        //d = up; // not strictly necessary :-)
        ++y;
    }
    dotline.dot1.col = x - 1;
    dotline.dot1.row = y - 1;
    dotline.dot2 = dotline.dot1;
    if (d == left)
        dotline.dot2.row++;
    else // d == up
        dotline.dot2.col++;
}
```

InitDots

/* Play begins with a call to your InitDots routine, where you are given the size of the game board (boardSize), an indicator of who plays first (playFirst), and a pointer to a CWindow (passed as a WindowPtr because that's what most toolbox routines expect). In that window, you will be required to display the progress of the game as it proceeds. */

```
void InitDots(
    short boardSize, // number of dots per row/col in board
    Boolean /* playFirst */, // true if you play first, false of opponent plays first
    WindowPtr dotWindow // color window where you should draw game results
) {
    #pragma unused(dotWindow)

    gCurrentPlayer = firstplayer; // whose turn it is to play
    gBoardSizeX = gBoardSizeY = boardSize - 1;
    Initialize();
    InitialDrawing();
}
```

OpponentMove

/* After your opponent has played, your OpponentMove routine will be called one or more times, once for each move made by your opponent. The move will be provided in the opponentLine parameter, for use in display and in updating your data structures. */
/* After each of your moves, and after notification of each opponent move, you should display the move and the updated game state in the dotWindow. The window should also display the number of squares completed by each player. The details of the display are left to you, as long as the display is correct. */

```
void OpponentMove(
    const DotLine opponentLine
    // line formed by your opponent on previous move
) {

    MoveRecordType move;
```

```

    ConvertFromDotLine(move, opponentLine);
    if (!MakeRealMove(move, gCurrentPlayer) &&
        gTotalScore != gMaxScore)
        NextPlayer(gCurrentPlayer);
}

```

PlayDots

```

/* When it is your turn to move, your PlayDots routine will be
called. Your code should select the most advantageous move and
return it in yourLines[0]. If that move forms a square, you can
select an additional move, store it in yourLines[1], and continue
as long as squares are formed. PlayDots should return the number
of moves you made during your turn. */
short /* number of lines generated */ PlayDots(
    DotLine yourLines[] // return the lines you form here
) {

```

```

    Boolean madeBox;
    MoveRecordType move;
    int moves = 0;
    do {
        ComputerTurn(move);
        madeBox = MakeRealMove(move, gCurrentPlayer);
        ConvertToDotLine(move, yourLines[moves]);
        ++moves;
    } while (madeBox && (gTotalScore != gMaxScore));

    if (gTotalScore != gMaxScore)
        NextPlayer(gCurrentPlayer);
    return moves;
}

```

TermDots

```

/* When all of the squares have been formed, your TermDots routine
will be called. You should deallocate any dynamically allocated
memory and perform any other cleanup required. */
void TermDots() { // return any storage you allocated

```

```

    delete [] gBoxes;

    for (int i=0; i<3; ++i)
    {
        PtrToTwoPathRecord p = gTwoPaths[i];
        while (p)
        {
            PtrToTwoPathRecord p2 = p->next;
            delete p;
            p = p2;
        }
    }
}

```

```

// Screen.cpp
// Copyright 2001 Jeff Mallett. All rights reserved.

```

```

#include <stdio.h>
#include <string.h>

#include <QuickDraw.h>

```

DRAWING CONSTANTS

```

// The full board can't actually fit on the window. I get less than 20x20
// boxes showing in the test code window. Therefore only draw
// the upper-left portion of grid if it's that big.
// The maximum dimensions to draw can be adjusted here if the window
// size is increased:
const int MAX_X_DRAW = 25;
const int MAX_Y_DRAW = 25;

```

```

const int BOX_WIDTH = 15;           /*pixel distance between adjacent dots*/
const int SCORE_TITLE_Y = 20;
const int SCORE_Y = 38;             /*distance of scores from top of screen*/
const int GRID_Y = 50;
const int SCORE_X[2] = {40, 130};
const char PLAYER_INITIAL[2] = { '1', '2' };
/*PLAYER_INITIAL[w] is the initial of player w*/

```

```

enum direction {left=0, up, right, down};
enum who {firstplayer=0, secondplayer};

```

```

extern int gBoardSizeX;
extern int gBoardSizeY;

```

PROTOTYPES

```

void ScreenLoc (int x, int y, int& i, int& j);
void DrawEdge (int x1, int y1, int x2, int y2);

```

```

void DrawDots();
void DrawScore (who w);
void DrawInitial (int x, int y, who person);
void DrawMove(int x, int y, direction d);
void DrawScoreTitle(who person);
void InitialDrawing();

DRAWING PROCEDURES
/*Given box coordinates x,y returns coords of top-left point of box (i,j)*/
void ScreenLoc (int x, int y, int& i, int& j) {

    i = x * BOX_WIDTH;
    j = y * BOX_WIDTH + GRID_Y;
} /*ScreenLoc*/

/*Given the coordinates of two boxes x1, y1 and x2, y2, this will draw in*/
/* an edge connecting the dot in the upper left hand corner of the two boxes.*/
void DrawEdge (int x1, int y1, int x2, int y2) {

    int i1, j1, i2, j2;
    ScreenLoc(x1, y1, i1, j1);
    ScreenLoc(x2, y2, i2, j2);
    MoveTo(i1, j1);
    LineTo(i2, j2);
} /*DrawEdge*/

/*Print the dots on the screen which form the playing board*/
void DrawDots() {

    int hor, ver, i, j;
    Rect rect;
    int maxh = gBoardSizeX;
    int maxv = gBoardSizeY;

    if (maxh > MAX_X_DRAW)
        maxh = MAX_X_DRAW;
    if (maxv > MAX_Y_DRAW)
        maxv = MAX_Y_DRAW;
    ++maxh;
    ++maxv;

    for( hor = 1; hor <= maxh; hor++)
        for( ver = 1; ver <= maxv; ver++) {
            ScreenLoc(hor, ver, i, j);
            SetRect(&rect, i, j, i + 1, j + 1);
            PaintOval(&rect);
        }
} /*DrawDots*/

/*Update a player's score on the drawing window*/
void DrawScore (who w) {

    extern int gScore[];
    int n;
    Rect rect;

    n = SCORE_X[w];
    SetRect(&rect, n, SCORE_Y - 11, n + 40, SCORE_Y + 2);
    EraseRect(&rect);

    char s[256];
    sprintf(s, "%1d", gScore[w]);

    MoveTo(n, SCORE_Y);
    DrawText(s, 0, strlen(s));
} /*DrawScore*/

void DrawInitial (int x, int y, who person) {

    if (x <= MAX_X_DRAW && y <= MAX_Y_DRAW)
    {
        int hor, ver;

        ScreenLoc(x, y, hor, ver);
        hor += (BOX_WIDTH / 2) - 3;

        ver += (BOX_WIDTH / 2) + 6;
        //TextFace(bold);
        MoveTo(hor, ver);
        DrawText(&PLAYER_INITIAL[person], 0, 1);
        //TextFace(normal);
    }
} /*DrawInitial*/

void DrawMove(int x, int y, direction d) {

    if (x <= MAX_X_DRAW && y <= MAX_Y_DRAW)

```

```

{
    switch (d) {
        case left :
            DrawEdge(x, y, x, y + 1);
            break;
        case up :
            DrawEdge(x, y, x + 1, y);
            break;
        case right :
            DrawEdge(x + 1, y, x + 1, y + 1);
            break;
        case down :
            DrawEdge(x, y + 1, x + 1, y + 1);
            break;
    }
}

void DrawScoreTitle(who person) {

    Rect rect;
    int n = SCORE_X[person];
    SetRect(&rect, n, SCORE_TITLE_Y - 11, n + 40,
        SCORE_TITLE_Y + 2);

    EraseRect(&rect);

    MoveTo(n, SCORE_TITLE_Y);
    DrawText(&PLAYER_INITIAL[person], 0, 1);
}

void InitialDrawing() {

    DrawScoreTitle(firstplayer);
    DrawScoreTitle(secondplayer);

    int x = SCORE_X[firstplayer]-10;
    int y = SCORE_TITLE_Y+4;
    int x2 = SCORE_X[secondplayer]+20;
    MoveTo(x, y);
    LineTo(x2, y);
    MoveTo((x+x2)/2, SCORE_TITLE_Y-15);
    LineTo((x+x2)/2, SCORE_Y+15);

    DrawDots();
}

```

Showcase



THIS MONTH'S ISSUE



[Issue 353](#)

[Code Debugging and Spelunking](#)

[Check out the latest including: x86 Code Debugging and Spelunking.; An Introduction to WebGL.; Managing ESXi from your Mac.; Deploying SHA-3.; An introduction to Julia.; KoolTools: BBEdit 11.5.; TextSoap For OS X.; Honeywell Lyric Round Wi-Fi Thermostat.; My Cloud EX2 Ultra.; MacTech Spotlight: Daniel H Steinberg](#)

See Now



Lost Data?
We Can Save it!

- Fast
- Reliable
- Certified
- Secure

800.440.1904
www.drivesavers.com

TODAY'S DEAL

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to PROACTIVE support and be the superhero your users expect

WATCHMAN
MONITORING



Search



Community Search:

GO

MacTech Search:

GO

Software Updates via MacUpdate



Latest Forum Discussions

SEE ALL

iPHONE APP SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

- SPREAD THE WORD:



- [Slashdot](#)



- [Digg](#)



- [Del.icio.us](#)



- [Reddit](#)



- [Newsvine](#)

- Generate a short URL for this page:

GET LINK

MacTech Magazine. www.mactech.com

Toll Free 877-MACTECH, Outside US/Canada: 805-494-9797

MacTech is a registered trademark of Xplain Corporation. Xplain, "The journal of Apple technology", Apple Expo, Explain It, MacDev, MacDev-1, THINK Reference, NetProfessional, Apple Expo, MacTech Central, MacTech Domains, MacNews, MacForge, and the MacTutorMan are trademarks or service marks of Xplain Corporation. Sprocket is a registered trademark of eSprocket Corporation. Other trademarks and copyrights appearing in this printing or software remain the property of their respective holders. Not responsible for typographical errors.

All contents are Copyright 1984-2011 by Xplain Corporation. All rights reserved. Theme designed by [Icreon](#).